
Weave — Audit démo ↔ produit

Où en est-on réellement, et que faut-il pour un pilote sérieux avec PennyLane

Guillaume Flambard

Juillet 2026

Verdict en une ligne

Ce qui est réellement construit (et vérifié)

Matrice de maturité par sous-système

Ce qui est réellement « pluggable » aujourd’hui

Les raccourcis assumés (pour ne pas être pris de court)

Ce qu’un ingénieur PennyLane va sonder — et la réponse

Chemin vers un pilote sérieux

Recommandation

Verdict en une ligne

Weave est aujourd'hui un « **walking skeleton** » **solide** : un système qui tourne **de bout en bout sur données rejouées**, prouve la thèse centrale (skills et agents émergent du travail réel), et dont l'**architecture est réellement pluggable**. Ce n'est pas encore un produit : il manque les connecteurs réels, l'auth/multi-tenant, un vrai modèle d'embeddings, et l'exploitation à l'échelle.

Niveau de maturité : ~2,5 / 5.

0 Idée Production	1 Spike	2 Walking skeleton	3 MVP données réelles	4 Pilote-ready	5
▲ NOUS (2,5)					
		tourne E2E, données seedées	1 tenant, 1 connecteur réel	auth, isolation, observabilité, SLA	

Traduction pour la décision : **c'est suffisant pour engager une conversation sérieuse et lancer un pilote cadré**, pas pour vendre un produit fini. L'atout est que le risque *conceptuel* est levé — reste du risque *d'ingénierie*, qui est prévisible et chiffrable.

Ce qui est réellement construit (et vérifié)

- **Pipeline event-sourcé** : ingestion idempotente, extraction faits/entités/ relations, embeddings, détection de schémas, matérialisation de skills. Vérifié bout en bout.
- **Émergence de skills** : une compétence naît d'un besoin récurrent, avec référents et sources. Vérifié (heuristique **et** LLM local Ollama).
- **Émergence d'agents + orchestration** : un spécialiste naît d'un cluster de skills, reste en attente d'approbation humaine, puis reçoit des délégations bornées (profondeur, nombre, temps) avec trace. Vérifié dans l'UI.
- **Recherche hybride + provenance** : vecteurs (pgvector) + plein-texte (tsvector), réponse avec couches mémoire affichées.
- **Multi-provider** : Ollama (local, sans clé) / Claude / heuristique, derrière un même port. Vérifié sur `qwen3.5:9b`.
- **MCP** : un agent externe interroge la mémoire via un protocole standard.
- **Dashboard** temps réel (SSE) au design Notion × PennyLane.

Matrice de maturité par sous-système

Légende : ● réel / prêt-prod · ● fonctionne mais qualité démo · ● design seulement.

Sous-système	État réel aujourd'hui	Niveau	Écart au produit
Event store & dédup	Postgres, immuable, hash de dédup	🟢	Partitionnement/rétention à l'échelle
Connecteurs (sources)	replay seedé ; trait Connector prêt	🔴	Le plus gros écart : OAuth Slack/GitHub/Notion réels, backfill, rate-limits
Extraction LLM	multi-provider, structured output, fallback	🟡	Harnais d'éval qualité, garde-fous de schéma, coût/latence
Embeddings	embedder lexical local (hachage)	🟡	Raccourci connu : brancher un vrai modèle (nomic/BGE/Voyage)
Vecteurs / BM25 / graphe	pgvector + tsvector + arêtes SQL	🟡	Mono-nœud ; passage Qdrant/Tantivy/Kuzu défini mais non fait
Émergence de skills	seuil d'occurrences + synthèse	🟢	Réglage des seuils, gouvernance, dédup sémantique
Émergence d'agents	cluster de skills → agent pending	🟡	Classifieur de domaine par mots-clés (à passer en embeddings)
Orchestration	plan→délègue→vérifie, garde-fous durs	🟡	Vérificateur heuristique → LLM adversarial ; budget tokens réel
Couches mémoire / provenance	4 niveaux, provenance affichée	🟡	Attribution du niveau par heuristique simple
API REST / SSE / MCP	complète, fonctionnelle	🟡	Aucune auth ; rate-limiting ; pagination
Versioning / time-travel	schéma le supporte (superseded_by)	🟡	Détection de contradiction non branchée, pas d'UI d'historique
Frontend	dashboard mono-projet	🟡	Multi-projet, états vides/erreurs, responsive, i18n
Sécurité / multi-tenant / RBAC	isolation par project seulement	🔴	Écart produit majeur : auth, tenants, RBAC, chiffrement, PII
Observabilité	logs tracing	🔴	OpenTelemetry / Prometheus / Grafana

Sous-système	État réel aujourd'hui	Niveau	Écart au produit
Tests / CI	tests unitaires (core, llm, ingest)	●	Tests d'intégration DB, e2e, pipeline CI, éval de régression
Déploiement	docker-compose + binaires locaux	●	Image prod, secrets, migrations gérées, K8s

Ce qui est réellement « pluggable » aujourd'hui

À dire sans exagérer face à un ingénieur PennyLane :

- **Le provider LLM se change par variable d'env** (`WEAVE_LLM_PROVIDER`) — Ollama, Claude, ou un adaptateur à écrire (OpenAI/Mistral) en ~1 fichier.
- **Le stockage est derrière des ports** (`EventStore` , `FactStore` , `VectorIndex` , `GraphStore` , `SkillStore` , `AgentStore`) : Qdrant/Kuzu/NATS = nouveaux adaptateurs, **pas** de réécriture du cœur.
- **Les agents externes se branchent via MCP** : Claude, un IDE, un autre orchestrateur interrogent la mémoire tout de suite.
- **Une nouvelle source se branche via le trait Connector** : le point d'ancrage existe, il reste à écrire l'adaptateur OAuth réel.

Nuance honnête : « pluggable » = *l'architecture le permet proprement*. Certains plugs sont **config** (provider LLM), d'autres demandent **un adaptateur à coder** (connecteur Slack, Qdrant). Ce n'est pas encore « installez et branchez ».

Les raccourcis assumés (pour ne pas être pris de court)

Un bon ingénieur les verra ; autant les nommer d'emblée, chacun a un chemin clair.

1. **Embedder lexical** (hachage n-gram) au lieu d'un modèle sémantique. → Brancher `nomic-embed-text` via Ollama (déjà local) : ~½ journée.
2. **Classifieur de domaine par mots-clés** pour l'émergence d'agents. → Clustering par embeddings sur les skills : ~2-3 jours.
3. **Vérificateur d'orchestration heuristique** (pas d'appel LLM). → Vérificateur LLM adversarial : ~1 jour.
4. **Signature de schéma ancrée sur un indice de thread** (l'équivalent d'un `thread_ts` Slack). C'est architecturalement sain, mais à documenter comme tel.
5. **Mono-tenant, sans auth.** → Le plus gros chantier produit (voir Phase 1).
6. **Données rejouées, pas de connecteur réel.** → Phase 0.

Ce qu'un ingénieur PennyLane va sonder — et la réponse

- « *C'est un wrapper de RAG ?* » → Non : c'est event-sourcé, ça distille des **faits versionnés**, et ça fait **émerger** skills et agents. Le RAG ne fait que retrouver des documents.
- « *Ça hallucine des skills ?* » → L'émergence est déclenchée par une **répétition mesurée** (seuil), la synthèse cite ses **sources** et **référénts**, et un agent émergent exige une **validation humaine**.
- « *Ça tient la charge ?* » → Mono-nœud aujourd'hui ; chemin d'échelle défini (Qdrant/Kuzu/NATS) sans réécriture grâce aux ports.
- « *Nos données sensibles ?* » → Isolation par projet aujourd'hui ; auth + multi-tenant + PII redaction = Phase 1, avant toute donnée réelle.
- « *Verrouillage fournisseur IA ?* » → Aucun : provider-agnostic, tourne même 100 % local (Ollama).

Chemin vers un pilote sérieux

Objectif : faire tourner Weave sur une **tranche sandboxée de données PennyLane réelles**, en sécurité, pour prouver la valeur sur leur terrain.

Phase 0 — « Réel sur un fil » (≈ 1 semaine) - Brancher un vrai modèle d'embeddings (Ollama `nomic-embed-text`). - Un connecteur **Slack en lecture seule** sur un canal de test. - Un mini-harnais d'éval (qualité d'extraction, précision d'émergence). - *Livrable : la même démo, mais sur de vrais messages Slack de test.*

Phase 1 — « Pilote-ready » (≈ 3-4 semaines) - **Auth + isolation multi-tenant** (RLS Postgres), redaction PII à l'ingestion. - Clustering d'agents par embeddings + vérificateur LLM adversarial. - Détection de contradictions + historique (time-travel). - Observabilité (OTel/Prometheus/Grafana) + CI (intégration + e2e). - *Livrable : un environnement isolé où PennyLane connecte un espace réel cadré.*

Phase 2 — « Passage à l'échelle » (≈ 2-3 mois) - Stockage scale-out (Qdrant/Kuzu/Tantivy), bus NATS JetStream. - Connecteurs GitHub/Notion/Linear, UI de gouvernance skills→agents. - Trajectoire sécurité (SOC 2), déploiement K8s.

Recommandation

Oui, c'est le bon moment pour engager une conversation sérieuse — mais avec le bon cadrage. Ne pas présenter Weave comme un produit ; le présenter comme ce qu'il est : un **runtime fonctionnel qui prouve une thèse que personne d'autre ne montre**, plus une spec technique, plus un plan de pilote crédible.

L'offre concrète à faire à PennyLane :

*« Voici un runtime de mémoire cognitive qui tourne, avec émergence de skills et d'agents, provider-agnostic (100 % local possible), et natif MCP. Voici la spec. Donnez-moi **2 semaines et un canal Slack de test** : je vous montre les mêmes émergences sur **vos** données, en environnement isolé. Si ça vous parle, on cadre un pilote. »*

C'est un point de départ bien plus fort qu'un deck : un **prototype qui marche** dé-risque la conversation et vous met en position de co-construction, pas de démarchage.